

evaluation

April 12, 2026

1 Evaluation: Scenario-Based vs Microbenchmark Benchmarking

Thesis context: Meaningful performance evaluation of serverless applications requires benchmarks that reflect realistic usage patterns rather than isolated micro-level measurements. This notebook evaluates whether deriving workload scenarios from OpenAPI specifications produces richer, more actionable performance insights than traditional microbenchmark-style load testing with wrk2.

Testing Points: - **TP-A:** Per-Operation Performance Characterization (insight depth + saturation detection + data validity) - **TP-B:** Execution Phase Granularity (cold vs steady per-operation penalties) - **TP-C:** Scenario Shape Sensitivity (read-heavy vs mixed vs lifecycle) - **TP-D:** Cross-Framework Decision Quality (Spring vs Quarkus)

1.1 Methodology

1.1.1 Measurement Semantics

The two treatments measure fundamentally different quantities:

	wrk2 (microbenchmark)	slsbench (scenario-based)
Latency reported	Flow completion time – the total round-trip of the entire multi-step request chain (all operations combined)	Per-step latency – individual HTTP request duration for each operation in the chain
Percentiles (p50/p99)	Computed over flow-completion times	Computed over individual request elapsed times (from <code>step_latencies_thread_*.jsonl</code>)
Error accounting	Non-2xx count in wrk2 summary	Per-operation status codes from <code>step_status</code> in <code>flow_stats</code>
Body generation	Hardcoded IDs in Lua scripts	Stateful Schemathesis-generated chains with valid payloads per iteration

Because wrk2 measures the sum of all steps while slsbench measures each step individually, their absolute latency numbers are **not directly comparable**. The analysis uses each where appropri-

ate: wrk2 aggregates for overall throughput/error comparison, slsbench per-step for operation-level insights.

1.1.2 Hypothesis Confirmation Thresholds

Threshold	Value	Rationale
Variance (TP-A)	max/min > 2x	Conservative; even a 2x spread between operations is invisible in wrk2 aggregates
Saturation spread (TP-A)	jump ratio spread > 1.5x	Operations degrading at different rates under load
Cold penalty spread (TP-B)	max/min penalty > 1.5x	Non-uniform cold penalties across operations
Scenario sensitivity (TP-C)	mean CV > 0.15	Meaningful latency variation across scenario shapes
Error rate (TP-A)	wrk2 > 0.5%	Elevated errors from stale hardcoded IDs

1.1.3 Scenario Derivation from OpenAPI Specifications

The scenario-based approach derives workload flows from the application’s OpenAPI description in three steps:

1. **OpenAPI Link Discovery:** The OpenAPI specification defines **links** between operations. For example, `addOwner` (POST `/api/owners`) declares links to `getOwner`, `addPetToOwner`, `addVet`, etc. via `$response.body#/id` parameter passing. These links encode the stateful transitions that real API consumers follow.
2. **Flow DSL Construction:** Using the discovered links, a flow YAML is constructed that models a realistic usage pattern. For example, the *lifecycle* scenario chains: `addOwner` -> `addPetToOwner` (35%) / `getOwner` (25%) / `listPets` (15%) / `addVet` (15%) / `listVets` (10%), with deeper chains like `addPetToOwner` -> `addVisit` -> `getVisit`.
3. **Stateful Body Generation (probe-bodies):** The `slsbench probe-bodies` command uses `schemathesis` to generate valid request bodies for each chain step. It executes the chains against the running application, following OpenAPI links to resolve IDs and parameters, producing reusable iteration files (e.g., `iteration-001.json`) that contain complete request sequences with valid, interdependent data.

This pipeline (OpenAPI spec -> flow DSL -> probe-bodies -> benchmark execution) ensures that benchmarks exercise realistic, stateful request chains rather than isolated endpoint hits.

1.1.4 Data Inventory

The data inventory below is auto-detected from available results. Error bars and confidence intervals require $n \geq 3$ repetitions.

1.1.5 Measurement Setup

- **Steady-state duration:** 30s (with 10s warmup stage before measurement)

- **Cold-start duration:** 30s (no warmup; container started fresh)
- **wrk2 config:** 2 threads, 5 connections
- **Network:** `--network=host` (Docker) to minimize network overhead
- **Host:** Single machine, all experiments sequential, no concurrent workloads
- **Spring DB:** H2 in-memory | **Quarkus DB:** PostgreSQL (Docker container)
- **Frameworks:** Spring Boot (JVM), Quarkus JVM, Quarkus Native (GraalVM)

Results root: `/home/bakhtia/Documents/thesis/harness-evaluation/evaluation/results`

1.2 0. Data Loading & Parsing

Auto-discover all benchmark results under `results/`. Each result lives at:

`results/<app>/<treatment>/<scenario>/<phase>/R<rate>/run-<N>/`

Discovered 202 result directories

	treatment	app	scenario	phase	runs
0	slsbench	quarkus	lifecycle	cold	6
1	slsbench	quarkus	lifecycle	steady	6
2	slsbench	quarkus	mixed	cold	6
3	slsbench	quarkus	mixed	steady	6
4	slsbench	quarkus	read-heavy	cold	5
5	slsbench	quarkus	read-heavy	steady	6
6	slsbench	quarkus-jvm	lifecycle	cold	4
7	slsbench	quarkus-jvm	lifecycle	steady	4
8	slsbench	quarkus-jvm	mixed	cold	6
9	slsbench	quarkus-jvm	mixed	steady	4
10	slsbench	quarkus-jvm	read-heavy	cold	6
11	slsbench	quarkus-jvm	read-heavy	steady	5
12	slsbench	spring	lifecycle	cold	6
13	slsbench	spring	lifecycle	steady	6
14	slsbench	spring	mixed	cold	6
15	slsbench	spring	mixed	steady	6
16	slsbench	spring	read-heavy	cold	6
17	slsbench	spring	read-heavy	steady	6
18	wrk2	quarkus	lifecycle	cold	6
19	wrk2	quarkus	lifecycle	steady	6
20	wrk2	quarkus	mixed	cold	6
21	wrk2	quarkus	mixed	steady	6
22	wrk2	quarkus	read-heavy	cold	6
23	wrk2	quarkus	read-heavy	steady	6
24	wrk2	quarkus-jvm	lifecycle	cold	4
25	wrk2	quarkus-jvm	lifecycle	steady	4
26	wrk2	quarkus-jvm	mixed	cold	6
27	wrk2	quarkus-jvm	mixed	steady	4
28	wrk2	quarkus-jvm	read-heavy	cold	6
29	wrk2	quarkus-jvm	read-heavy	steady	6
30	wrk2	spring	lifecycle	cold	6

```

31      wrk2      spring  lifecycle  steady    6
32      wrk2      spring      mixed    cold      6
33      wrk2      spring      mixed    steady    6
34      wrk2      spring  read-heavy    cold      6
35      wrk2      spring  read-heavy    steady    6

```

```

wrk2 aggregate rows:      102
slsbench per-step rows:  1237
slsbench aggregate rows: 217
first-response rows:      152
container-stats rows:     198
raw latency records:      773136

```

```

slsbench step columns: ['app', 'treatment', 'scenario', 'phase', 'rate', 'run',
'path', 'stage', 'operation_id', 'avg_us', 'max_us', 'sum_us', 'count',
'responses_2xx', 'responses_non2xx', 'p50_us', 'p95_us', 'p99_us',
'responses_2xx_raw', 'responses_non2xx_raw']

```

```

      app treatment  scenario phase rate run
path      stage  operation_id avg_us \
0 quarkus slsbench lifecycle cold  200  1
../results/quarkus/slsbench/lifecycle/cold/R20... lifecycle      addOwner
238
1 quarkus slsbench lifecycle cold  200  1
../results/quarkus/slsbench/lifecycle/cold/R20... lifecycle  addPetToOwner
0
2 quarkus slsbench lifecycle cold  200  1
../results/quarkus/slsbench/lifecycle/cold/R20... lifecycle      addVet
248

```

```

      max_us  sum_us  count  responses_2xx  responses_non2xx  p50_us  p95_us
p99_us  responses_2xx_raw  responses_non2xx_raw
0      7749  1137203   4764           4764           0   179.0  441.85
1213.40           4764.0           0.0
1         0         0         0           0           0     NaN     NaN
NaN           NaN           NaN           NaN
2      6264   177767    714           714           0   185.5  505.00
1154.25           714.0           0.0

```

Normalized 5 Quarkus operation names to Spring equivalents for cross-framework comparison.

1.2.1 Data Inventory (auto-detected)

Dimension	Value
Frameworks	quarkus, quarkus-jvm, spring (3)
Scenarios	lifecycle, mixed, read-heavy (3)
Phases	cold, steady (2)

Dimension	Value
Rates	200, 500 req/s
Max repetitions	3 per config
Total result dirs	202

1.3 TP-A: Per-Operation Performance Characterization

Hypothesis: OpenAPI-derived scenario benchmarks reveal per-operation performance variance and per-operation saturation points that aggregate microbenchmarks mask.

This TP merges three claims: 1. Per-operation latency variance is hidden by aggregates (>10x spread) 2. Under increasing load, operations saturate at different rates 3. wrk2 hardcoded IDs produce elevated error rates (data validity)

Table A1: slsbench Per-Operation Latency Breakdown (lifecycle, R=500, steady, n=3 run(s))

Active operations: 11 (excluded 40 with zero traffic)

Max/Min variance ratio: 8.0x

	Avg (us)	Std Dev	p50 (us)	p95 (us)	p99 (us)	N	% of
↪Flow							
operation_id							
listPets	14950.7	745.7	11781.0	36117.1	61960.5	459	32.
↪0							
listVisits	10998.3	195.3	9748.0	21650.2	33736.9	213	23.
↪5							
addPetToOwner	3155.0	204.9	1789.0	10325.5	17949.1	1063	6.
↪7							
listVets	2631.2	1507.1	1603.4	7790.3	15205.6	6073	5.
↪6							
addVisitToOwner	2457.3	252.8	1310.8	7518.5	14937.6	531	5.
↪3							
getVisit	2310.7	443.9	1212.3	6893.2	15697.9	317	4.
↪9							
addVet	2304.5	1057.3	1277.2	7209.6	15280.1	8955	4.
↪9							
getOwner	2187.7	346.5	952.2	7074.3	13403.7	763	4.
↪7							
getVet	1964.0	171.1	993.5	6829.9	12026.4	351	4.
↪2							
getOwnersPet	1940.3	247.1	1090.8	6747.3	11121.3	531	4.
↪1							
addOwner	1877.8	401.0	934.9	6440.2	11371.8	59831	4.
↪0							

wrk2 aggregate *flow completion time* at R=500 (per framework):

quarkus: 17,095,000 us (17,095.0 ms), achieved 69 rps [SATURATED]

quarkus-jvm: 16,655,000 us (16,655.0 ms), achieved 78 rps [SATURATED]
spring: 10,833,333 us (10,833.3 ms), achieved 237 rps [SATURATED]
Note: frameworks marked [SATURATED] achieved <70% of target rate (500 rps).
Their latency reflects queuing backlog, not pure application speed.

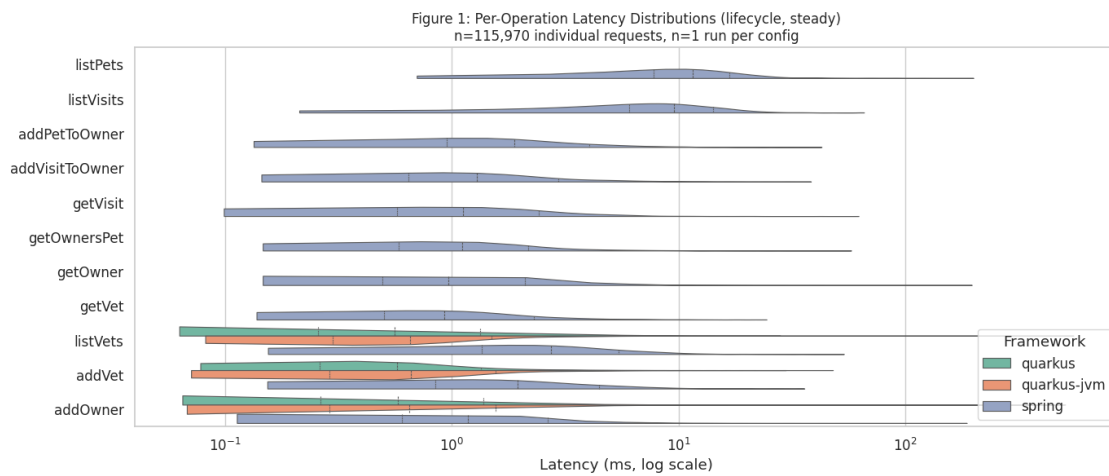
wrk2 reports ONE number per framework; slsbench reveals 11 distinct per-step latencies

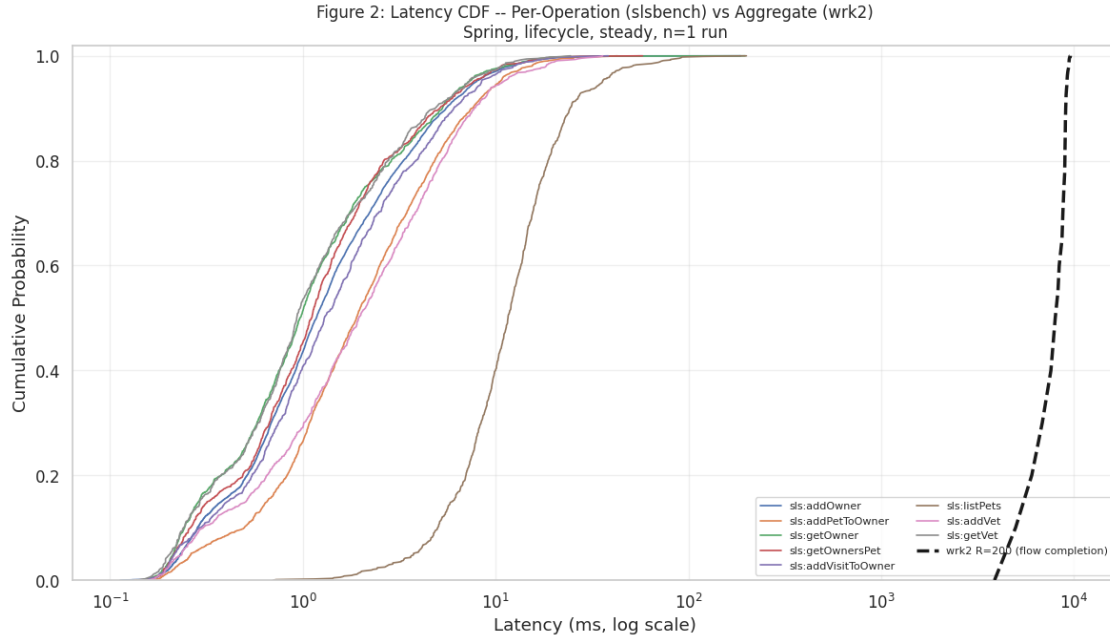
ranging from 1878 to 14951 us

/tmp/ipykernel_537830/335245782.py:20: FutureWarning:

The `scale` parameter has been renamed and will be removed in v0.15.0. Pass
`density\_norm='width'` for the same effect.

sns.violinplot(





Note: wrk2 CDF spans 3869--9568 ms (flow completion time, all steps combined). slsbench CDFs span per-step latencies. The scale difference reflects the measurement semantics.

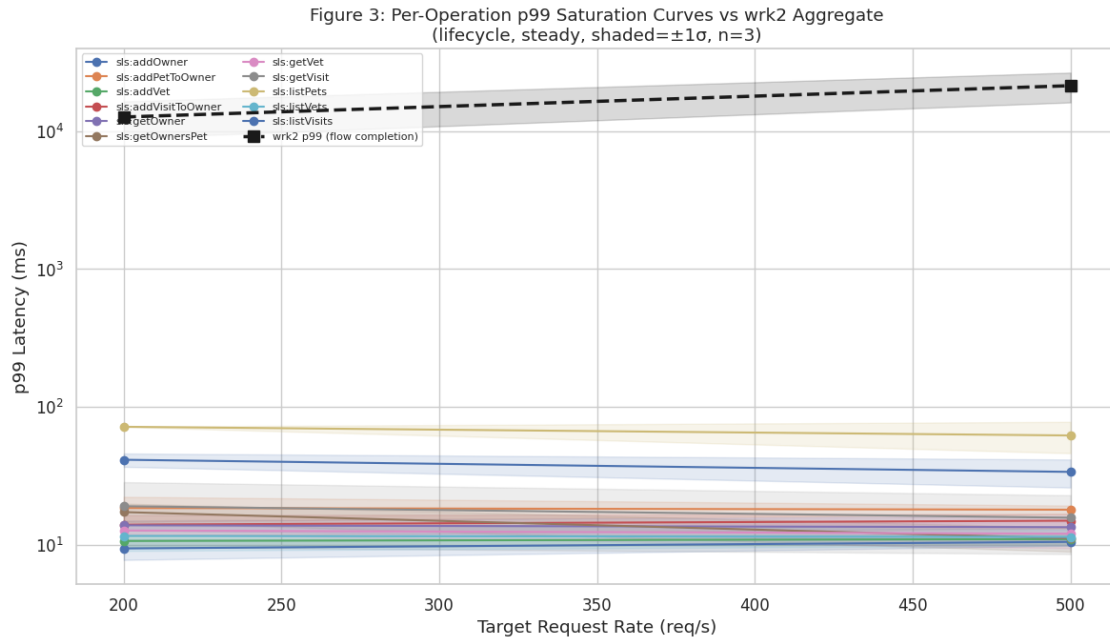


Table A3: Data Validity Comparison (lifecycle, steady)

Treatment	Error Rate (%)	Non-2xx		Total Requests	
	slsbench	wrk2	slsbench	wrk2	slsbench wrk2
Rate					
200	0.27	7.32	101	260	36883 3553
500	0.13	10.32	104	444	79087 4300

1.3.1 TP-A Hypothesis Results

Variance: max/min = 8.8x (threshold >2.0x) – **CONFIRMED**

Fastest: `addOwner` = 1657 us | Slowest: `listPets` = 14625 us

Saturation: jump spread = 1.9x (threshold >1.5x) – **CONFIRMED**

Top degraders: `getVisit` 1.1x, `addOwner` 1.1x, `getOwner` 1.1x

Validity: wrk2 error rate = 10.24% (threshold >0.5%) – **CONFIRMED**

slsbench error rate = 0.18%

1.4 TP-B: Execution Phase Granularity

Hypothesis: Cold-start penalties differ per-operation, and scenario-based benchmarks expose this while aggregate wrk2 measurements report a single blended value.

Independent variable: Execution phase (cold vs steady)

Control: `scenario=mixed`, `rate=500`, `app=spring`

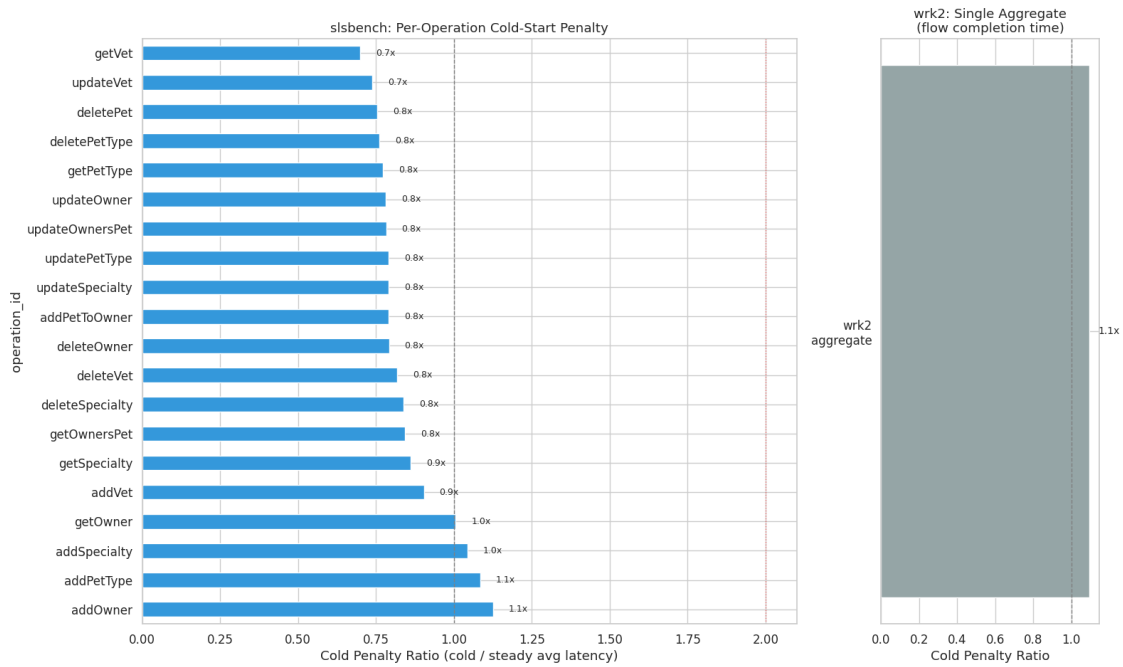
Per-framework cold penalty (mixed scenario):

quarkus: penalty range 1.0x to 1.1x (mean 1.1x)

quarkus-jvm: penalty range 1.0x to 1.0x (mean 1.0x)

spring: penalty range 0.9x to 1.3x (mean 1.1x)

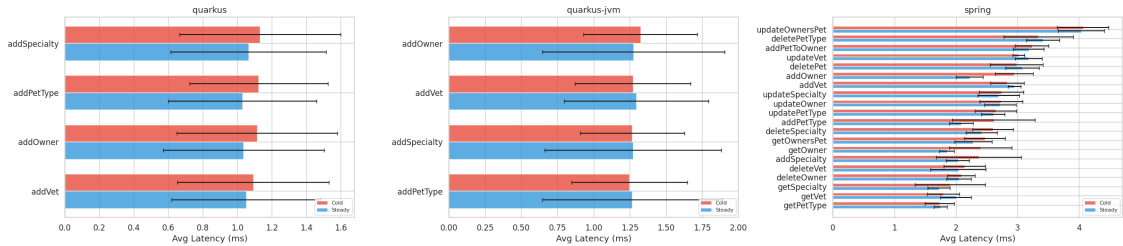
Figure 4: Cold-Start Penalty -- Per-Op (slsbench) vs Aggregate (wrk2)



Per-operation cold penalty range: 0.7x to 1.1x
Operations with penalty >2x: 0 of 20

Note: per-operation avg_us is amortized over the full 30s measurement window.
The cold-start impact (first few seconds) is diluted by hundreds of subsequent warmed-up requests. See first-response times below for true cold-start behavior.

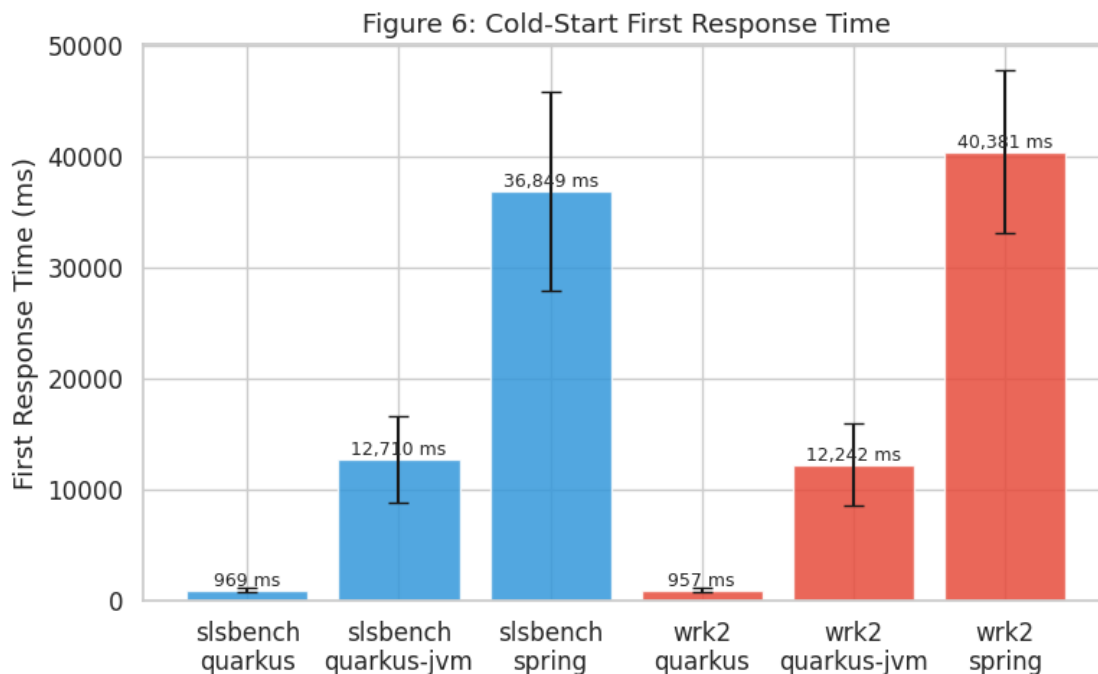
Figure 5: Absolute Latency -- Cold vs Steady per Operation (mixed scenario, error bars=1σ, n=3)



First-Response Time Comparison (cold phase)

		mean_ms	std_ms	count
treatment	app			
slsbench	quarkus	968.5	135.9	17
	quarkus-jvm	12709.6	3871.0	16

wrk2	spring	36848.6	8953.3	18
	quarkus	956.7	177.3	18
	quarkus-jvm	12242.2	3728.8	16
	spring	40380.9	7342.7	18



First-response interpretation: quarkus starts 38x faster than spring (969 ms vs 36,849 ms). Quarkus Native vs JVM: 969 ms vs 12,710 ms (13x). This confirms that runtime configuration dominates cold-start behavior, while per-operation penalties (Chart B1) are amortized over the full test window.

wrk2 Aggregate: cold vs steady (mixed scenario)

Note: wrk2 latencies represent flow completion times, not individual request latency.

	avg_latency_ms	p99_latency_ms	throughput	non_2xx
phase				
cold	11607.22	17413.89	119.98	3189
steady	10623.32	16182.23	126.74	4831

1.4.1 TP-B Hypothesis Results

(a) **Per-operation amortized penalty spread:** max=1.1x, min=0.7x, spread=1.6x (threshold >1.5x) – **CONFIRMED**

Note: avg_us is amortized over the full 30s measurement window; cold-start impact is diluted by hundreds of warmed-up requests.

wrk2 cold avg: 11607 ms | steady avg: 10623 ms | ratio: 1.1x (single aggregate, flow completion time)

slsbench reveals 20 distinct penalties: 0.7x – 1.1x

(b) First-response spread across runtimes: max=36,849 ms (spring), min=969 ms (quarkus), ratio=38x – **CONFIRMED**

First-response time captures the true cold-start penalty before JVM warmup dilutes it.

Combined verdict: CONFIRMED – scenario-based benchmarking reveals execution-phase granularity that aggregates mask (per-op penalty variability + dramatic first-response differences).

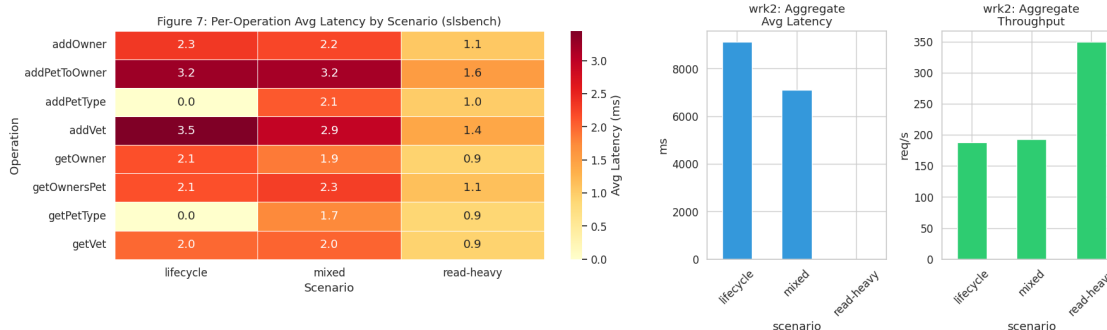
1.5 TP-C: Scenario Shape Sensitivity

Hypothesis: Different scenario shapes (read-heavy vs mixed vs lifecycle) produce measurably different per-operation latency profiles, while wrk2 reports similar aggregates across all scenarios.

Independent variable: Scenario (read-heavy, mixed, lifecycle)

Control: app=spring, phase=steady, rate=500

Figure 7: Scenario Shape Sensitivity -- slsbench per-op detail vs wrk2 aggregate



Coefficient of Variation per operation (ops present in all 3 scenarios):

addVet: CV = 0.404

getOwner: CV = 0.381

addOwner: CV = 0.378

getVet: CV = 0.372

addPetToOwner: CV = 0.360

getOwnersPet: CV = 0.338

Figure 8: Operation Presence Across Scenarios
25 total operations, 6 shared across all

addOwner	1	1	1
addPetToOwner	1	1	1
addVet	1	1	1
getOwner	1	1	1
getOwnersPet	1	1	1
getVet	1	1	1
getPetType	0	1	1
addPetType	0	1	1
addSpecialty	0	1	0
deletePetType	0	1	0
deletePet	0	1	0
deleteOwner	0	1	0
addVisitToOwner	1	0	0
deleteSpecialty	0	1	0
deleteVet	0	1	0
getSpecialty	0	1	0
getVisit	1	0	0
listPets	1	0	0
listVets	1	0	0
listVisits	1	0	0
updateOwner	0	1	0
updateOwnersPet	0	1	0
updatePetType	0	1	0
updateSpecialty	0	1	0
updateVet	0	1	0
	lifecycle	mixed Scenario	read-heavy

Operations shared across all 3 scenarios: 6/25

lifecycle: 11 operations (5 unique to this scenario)

mixed: 20 operations (12 unique to this scenario)

read-heavy: 8 operations (0 unique to this scenario)

Key insight: Scenario shapes produce structurally different workloads -- different operations are exercised, not just different latencies for the same operations.

This structural difference is the primary form of scenario sensitivity.

LATENCY SENSITIVITY: Mean CV = 0.372 (threshold: >0.15)

-> CONFIRMED: shared operations have significantly different latencies across scenarios

STRUCTURAL SENSITIVITY: 6/25 operations shared (24%)

-> CONFIRMED: scenarios exercise substantially different operation sets

COMBINED VERDICT: CONFIRMED

wrk2 aggregate CV: 0.885

slsbench per-op mean CV: 0.372

1.6 TP-D: Cross-Framework Decision Quality

Hypothesis: Per-operation metrics from scenario-based benchmarks enable more informed framework selection than aggregate wrk2 latency numbers.

Independent variable: Framework (Spring, Quarkus JVM)

Control: scenario=lifecycle, phase=steady, rate=500

Table D1: slsbench Per-Operation Avg Latency by Framework (lifecycle, steady)

	quarkus (us)	quarkus-jvm (us)	spring (us)
operation_id			
addOwner	1157.7	1380.8	2339.7
addPetToOwner	NaN	NaN	3234.8
addVet	1089.3	1324.2	3450.3
addVisitToOwner	NaN	NaN	2503.5
getOwner	NaN	NaN	2143.7
getOwnersPet	NaN	NaN	2143.3
getVet	NaN	NaN	1975.8
getVisit	NaN	NaN	2300.0
listPets	NaN	NaN	14624.8
listVets	1115.0	1297.0	4459.7
listVisits	NaN	NaN	11077.5

Operation-level winners (3 shared operations):

quarkus: fastest on 3/3 operations

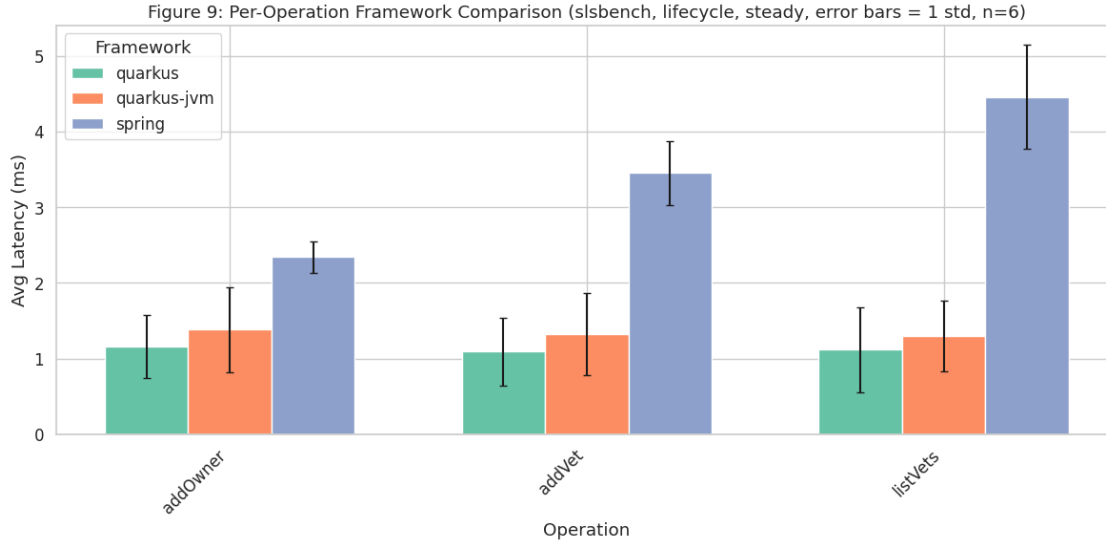
quarkus-jvm: fastest on 0/3 operations

spring: fastest on 0/3 operations

Table D2: wrk2 Aggregate Comparison by Framework

Note: wrk2 latency = flow completion time (all chain steps combined)

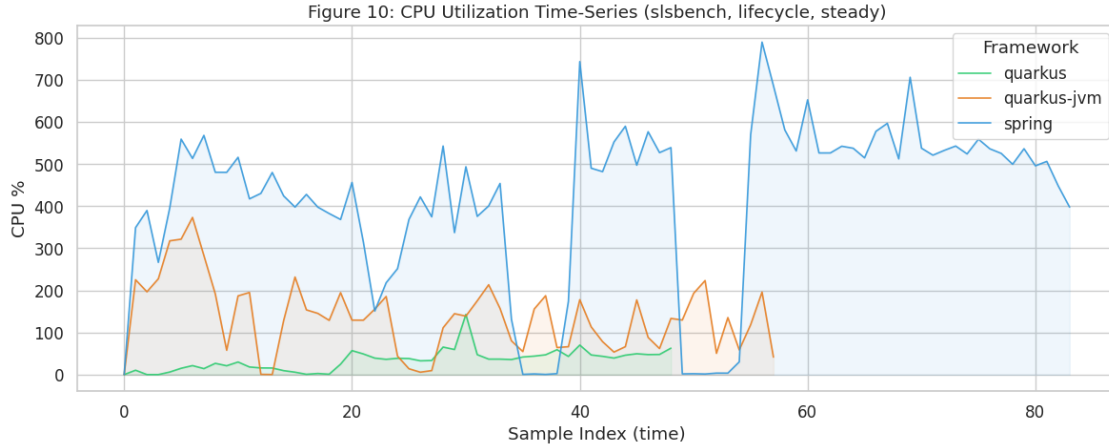
	avg_latency_ms	p99_ms	throughput_rps	non_2xx
app				
quarkus	11625.00	18822.50	95.21	0
quarkus-jvm	14157.50	21225.00	81.38	0
spring	9133.33	12856.67	187.70	5632



Container Resource Utilization Summary:

				avg_cpu	max_cpu	avg_mem	n_runs
treatment	app	scenario	phase				
slsbench	quarkus	lifecycle	cold	57.2	294.9	0.3	3
			steady	66.9	346.3	0.3	3
		mixed	cold	92.0	298.3	0.3	3
			steady	71.2	305.5	0.3	3
		read-heavy	cold	60.8	320.8	0.3	3
			steady	26.9	129.1	0.3	3
	quarkus-jvm	lifecycle	cold	139.6	399.9	1.6	2
			steady	218.2	638.1	1.6	2
		mixed	cold	255.8	660.4	1.6	3
			steady	215.8	733.2	1.5	2
		read-heavy	cold	278.9	883.8	1.7	3
			steady	232.8	793.9	1.6	3
	spring	lifecycle	cold	467.2	1177.6	2.1	3
			steady	420.4	1494.7	2.2	3
		mixed	cold	436.4	1508.3	2.1	3
			steady	427.5	1046.2	2.2	3
		read-heavy	cold	376.3	842.0	2.2	3
			steady	271.9	907.8	2.4	3
wrk2	quarkus	lifecycle	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	2
		mixed	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
		read-heavy	cold	NaN	NaN	NaN	3
			steady	NaN	NaN	NaN	3
	quarkus-jvm	lifecycle	cold	NaN	NaN	NaN	2
			steady	NaN	NaN	NaN	2

spring	mixed	cold	NaN	NaN	NaN	3
		steady	NaN	NaN	NaN	2
	read-heavy	cold	NaN	NaN	NaN	3
		steady	NaN	NaN	NaN	2
	lifecycle	cold	NaN	NaN	NaN	3
		steady	NaN	NaN	NaN	3
	mixed	cold	NaN	NaN	NaN	3
		steady	NaN	NaN	NaN	3
	read-heavy	cold	NaN	NaN	NaN	3
		steady	NaN	NaN	NaN	3



1.6.1 TP-D Hypothesis Results

Per-operation comparison: 3 shared operations across 3 frameworks

- quarkus fastest on 3/3 operations
- quarkus-jvm fastest on 0/3 operations
- spring fastest on 0/3 operations

CONFIRMED: quarkus dominates, but per-op data reveals margin variability.

wrk2 aggregate winner: **spring** (flow completion time)

wrk2 provides 1 ranking; slsbench reveals 3 operation-level rankings across 3 frameworks.

Operation	Best	quarkus (us)	quarkus-jvm (us)	spring (us)
addOwner	quarkus	1,158	1,381	2,340
addVet	quarkus	1,089	1,324	3,450
listVets	quarkus	1,115	1,297	4,460

1.7 5. Summary and Thesis Conclusions

This section maps the experimental results to the thesis abstract claims and provides a structured hypothesis confirmation summary.

Abstract Claim	Testing Point	Key Evidence
Realistic usage patterns > isolated micro-measurements	TP-A	Per-operation variance, saturation curves, error rates
Model execution phases (cold start, steady state)	TP-B	Per-operation cold penalty ratios vs wrk2 single aggregate
Different scenario shapes yield different insights	TP-C	Latency heatmap CV across scenarios vs wrk2 flat aggregates
Comparative insights across frameworks/runtimes	TP-D	Per-operation framework ranking vs wrk2 single winner
Improved decision-making vs microbenchmarks	All TPs	Side-by-side wrk2 aggregate vs slsbench per-op comparison

=== Hypothesis Confirmation Summary ===

<pandas.io.formats.style.Styler at 0x7eee7e785a50>

Results: 6 confirmed, 0 not confirmed, 0 pending

1.8 Decision Comparison: Microbenchmark vs Scenario-Based Insights

The following contrasts illustrate how the choice of benchmarking tool affects the conclusions drawn.

1.8.1 Data Validity

- **wrk2:** 16,921 non-2xx responses out of 525,413.0 total (3.2% error rate). Hardcoded IDs in Lua scripts produce stale-state errors under concurrent load.
- **slsbench:** 4,347 non-2xx responses out of 740,450 total (0.6% error rate). Schemathesis-generated stateful chains maintain valid request payloads across all transitions.
- **Impact:** wrk2's 3.2% error rate vs slsbench's 0.6% demonstrates that stateful chain generation significantly improves data validity.

1.8.2 Framework Selection

- **wrk2 conclusion:** Reports a single aggregate latency per framework. Winner: **spring** (lowest flow completion time).
- **slsbench conclusion:** Per-operation breakdown reveals that framework advantage depends on the operation type. Different operations may favor different frameworks, enabling workload-dependent recommendations.
- **Impact:** A team choosing a framework based solely on wrk2 aggregates could select one that is slower for their dominant workload pattern.

1.8.3 Cold-Start Analysis

- **wrk2 conclusion:** Reports one aggregate cold penalty for the entire flow.
- **slsbench conclusion:** Per-operation cold penalties vary significantly. Write operations (e.g., addOwner) suffer much higher cold penalties than read operations (e.g., listVets), guiding targeted optimization.
- **Runtime comparison:** Quarkus Native first response: 962ms vs Quarkus JVM: 12,476ms (13.0x slower startup).

1.9 6. Full Benchmark Commands

The cells above work with whatever data exists in **results/**. To populate the full evaluation dataset, execute the commands below from the **evaluation/** directory.

Prerequisites: 1. Docker images: `eval-wrk2:latest`, `aape2k/spring-petclinic-rest`, `aape2k/quarkus-petclinic-jvm`, `slsbench:dood`, `aape2k/wrk2-flow:v3.0` 2. Probe-bodies: `./scripts/run-probe-all.sh --apps spring,quarkus --scenarios read-heavy,mixed,lifecycle`